

資料庫導論

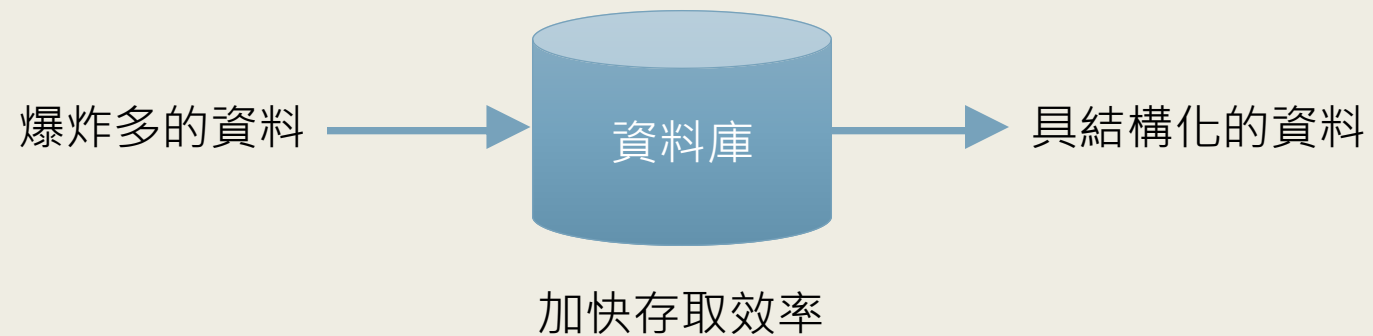
朱克剛

<https://www.chainhao.com.tw>



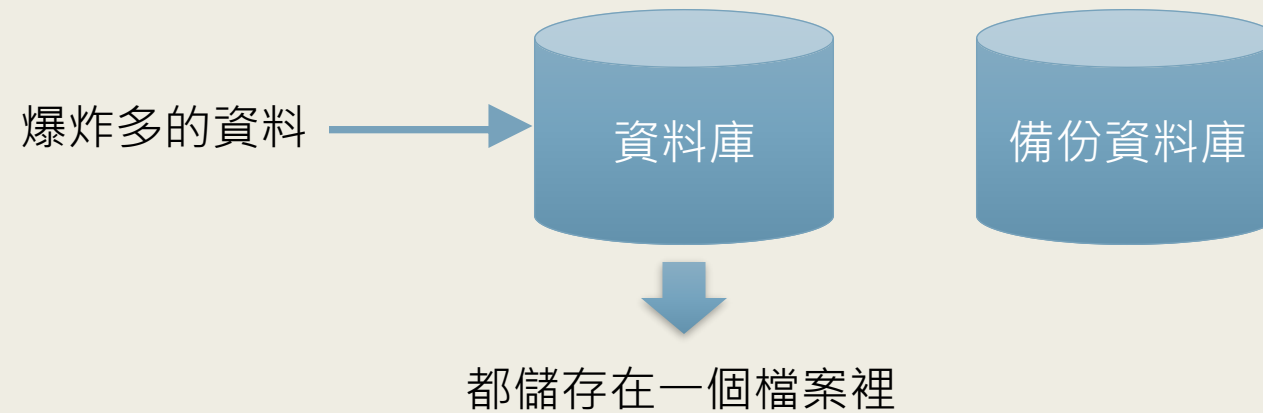
資料庫是什麼？

- 資料庫是一套專門用來管理資料的應用程式



雞蛋放在同一個籃子裡

- 資料庫的備份或備援非常重要



資料庫類型

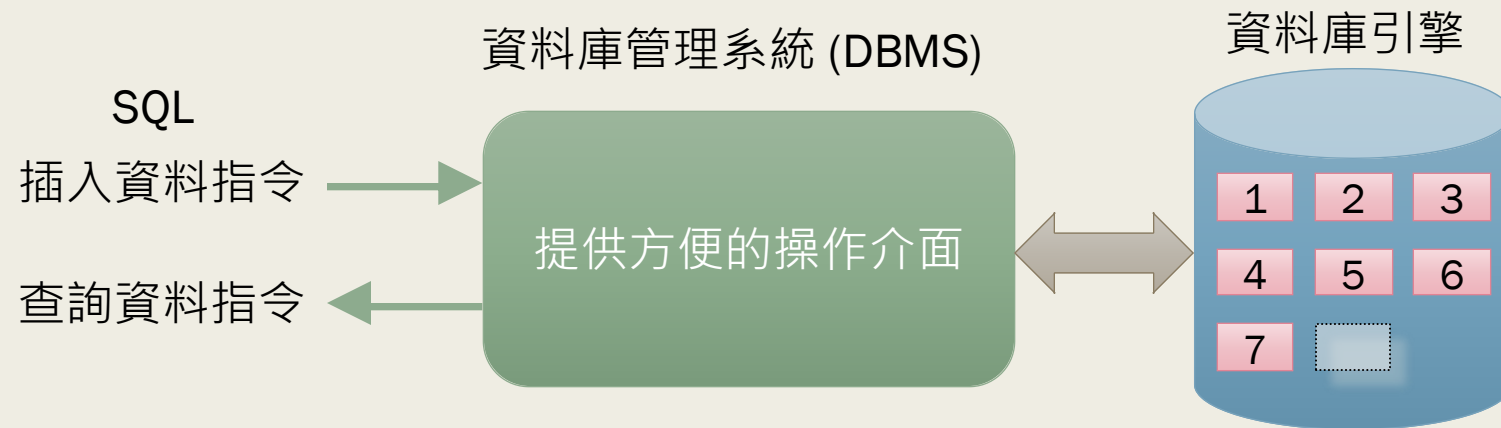
資料庫列表與排名

<https://db-engines.com/en/ranking>

- 關連式資料庫 (SQL)
 - 強調資料與資料間的關連性
 - 儲存空間小但規劃時間長
 - 需先將資料調整為特定結構後才能送進資料庫
 - 以 MySQL 、 SQL Server 、 Oracle 為代表
- 非關連式資料庫 (NoSQL)
 - 隨著資料類型不同，資料庫種類多元
 - 儲存空間需求通常不是關注重點，資料庫規劃時間短
 - 以 MongoDB (文本資料庫) 、 Neo4j (圖形資料庫) 為代表

資料庫引擎與管理系統

- 如果公司使用多個不同資料庫引擎，通常會使用第三方資料庫管理系統
- 與資料庫連接的後端應用程式相當於有特定功能的資料庫管理系統



SQL指令種類

- 資料定義語言 (DDL, data definition lang.)
 - Create: 建立資料庫物件
 - Alter: 變更資料庫物件
 - Drop: 刪除資料庫物件
- 資料控制語言 (DCL, data control lang.)
 - Grant: 設定權限
 - Revoke: 取消權限
- 資料操作語言 (DML, data manipulation lang.)
 - Insert Into: 插入資料
 - Update: 修改資料
 - Delete: 刪除資料
- 資料查詢語言 (DQL, data query lang.)
 - Select: 查詢資料
- 交易控制語言 (TCL, transaction control lang.)
 - Commit: 交易完成
 - Rollback: 交易失敗

好的DBMS可以很容易地完成這部分

要非常非常熟悉
後端全靠他
AI 可幫忙

在維持資料一致性
需求時必須使用，
例如搶票

關連式資料庫的組成

- 實體 (entity) -> 資料表 (table)
- 屬性 (attribute) -> 欄位 (field)
- 關連性 (relationship) -> 心中



儲存聯絡人的資料庫

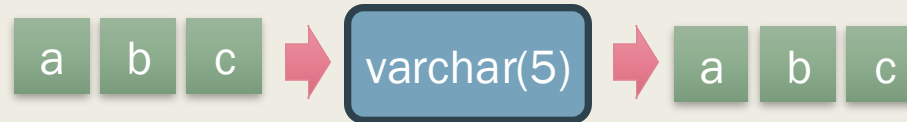
- 帳號 / 姓名 / 住址 / 電話

是不是很像 Excel

uid	cname	address	tel
A01	王大明	台中市台灣大道一段 50 號	1111
A02	李大媽	台中市台灣大道一段 50 號	1111
A03	王小毛	台中市台灣大道一段 50 號	1111

如何在資料庫儲存資料？

- 第一步：建立資料庫
- 第二步：建立資料表
- 第三步：建立欄位與決定欄位型態
 - 初學只要認識文字型態 (`char` 、 `varchar`) 與數字型態 (`int` 、 `float`) 即可
 - `char` : 固定長度字串
 - `varchar` : 可變動長度字串
- 第四步：開始儲存資料



只有一個資料表時的問題

- 資料大量重複造成容易出錯、維護困難且浪費儲存空間

uid	cname	address	tel
A01	王大明	台中市台灣大道一段 50 號	1111
A02	李大媽	台中市台灣大道一段 50號	1111
A03	王小毛	台中市台灣大道1段 50 號	1111

這是三個不同的地址，看的出來嗎？

將一個表切成兩個表

- 有比較好嗎？沒有，因為地址一樣會重複會出錯

uid	cname	address
A01	王大明	台中市台灣大道一段50號
A02	李大媽	台中市台灣大道一段50號
A03	王小毛	台中市台灣大道一段50號

address	tel
台中市台灣大道一段50號	1111

下一步 - 給每個地址一個唯一編號

- 當 hid 為數字型態時，只會有一種資料內容，不像文字型態，內容可以任意輸入
- 此外，儲存空間需求會比只有一個表時大量減少

uid	cname	hid
A01	王大明	1
A02	李大媽	1
A03	王小毛	1

hid	address	tel
1	台中市台灣大道一段50號	1111
2	台中市中山路10號	2222

1
01
001
1.0



1

—
①
1



✗

儲存空間需求

假設一個字佔
3bytes

uid	cname	address	tel
A01	王大明	台中市台灣大道一段 50 號	1111
A02	李大媽	台中市台灣大道一段 50號	1111
A03	王小毛	台中市台灣大道1段 50 號	1111

$$(20 + 50 + 100 + 30) * 3 * 3 = 1800 \text{ bytes}$$

uid	cname	hid
A01	王大明	1
A02	李大媽	1
A03	王小毛	1

hid	address	tel
1	台中市台灣大道一段50號	1111

$$4 + (100 + 20) * 3 = 364 \text{ bytes}$$

$$642 + 364 = 1006 \text{ bytes}$$

$$((20 + 50) * 3 + 4) * 3 = 642 \text{ bytes}$$

確保資料不可重複 - PK

- 如何確保左邊的 uid 與右邊的 hid 資料不重複？設定 PK (主鍵或稱主索引)
- 資料庫保證 PK 欄位內容不會重複以及不會跳過沒輸入資料
- 使用「底線」表示 PK

<u>uid</u>	cname	hid
A01	王大明	1
A02	李大媽	1
A03	王小毛	1
✗A01	朱小妹	2

<u>hid</u>	address	tel
1	台中市台灣大道一段50號	1111
2	台中市中山路10號	2222
✗NULL	高雄市五福路100號	NULL

資料表是否需要 PK

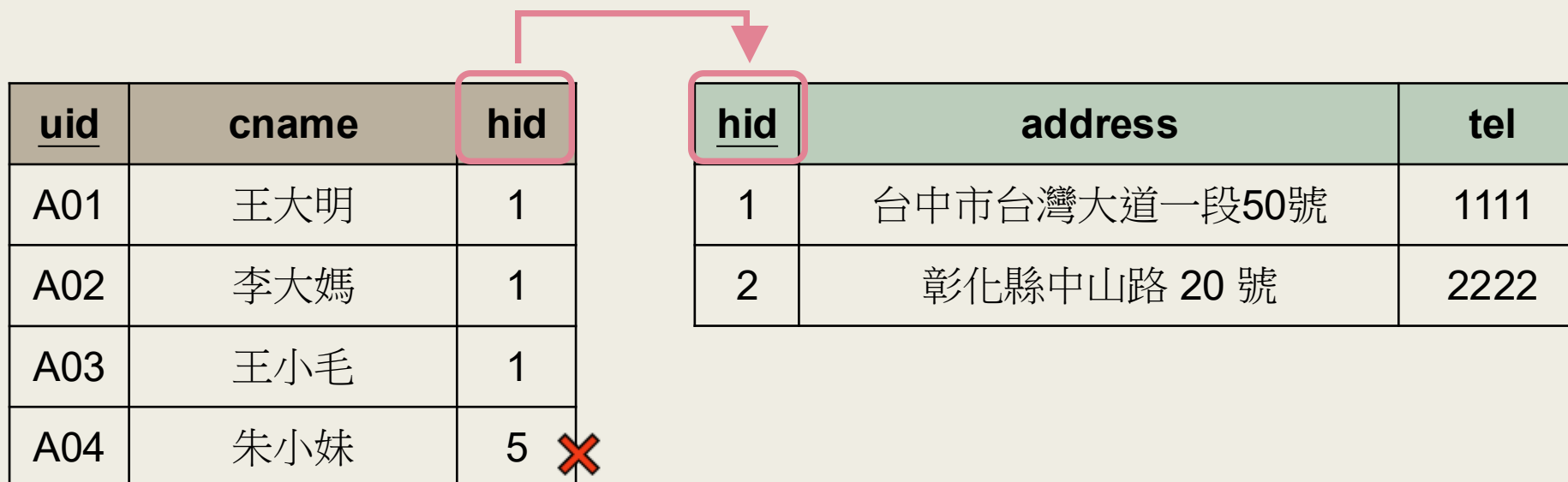
- 關連式資料庫每個資料表都應該要有 PK，而且只能有一個
- 一般都以最少且資料不重複的欄位為 PK
- PK 除了單一欄位外，也可以多個欄位組合起來，稱為複合欄位 PK
- 實在找不出適當的欄位當 PK 時，可以新增一個自動編號欄位作為 PK，此欄位型態必須是整數型態

<u>book</u>	<u>author</u>
哈利波特	JK羅琳

<u>id</u>	book	author
1	哈利波特	JK羅琳
2	...	...

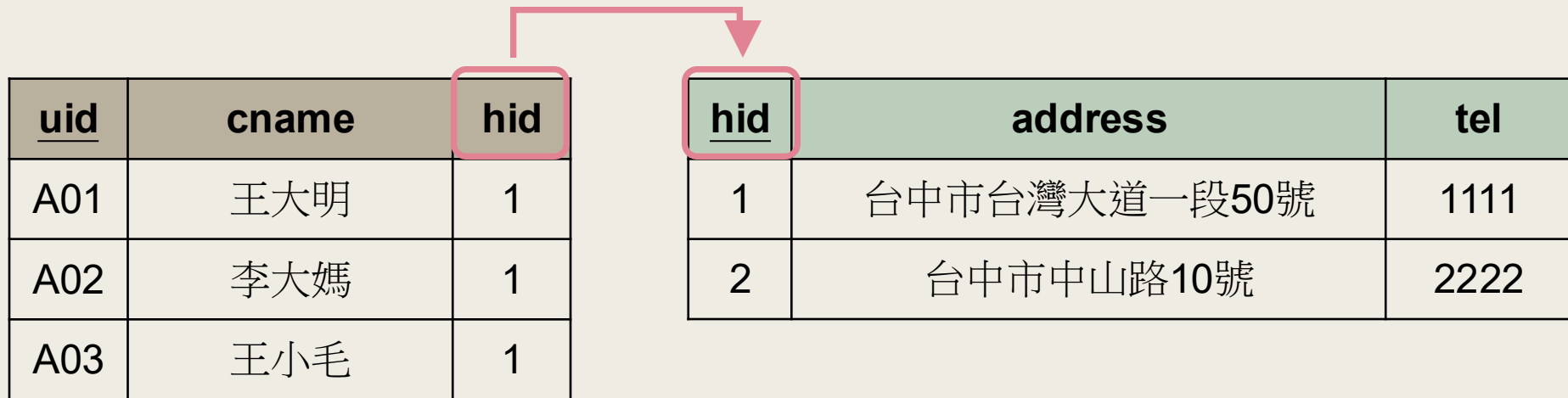
確保資料一定存在 - FK

- 如何確保左邊的 hid 資料一定是右邊有的？設定 FK（外來鍵又稱參考索引）
- 被參考的欄位必須是 PK 或是唯一索引（unique index）



FK 視作關連線

- 王大明的電話是幾號？看圖很自然的會說「1111」，因為 FK 已經幫我們將兩個表「連」起來了



FK 重要嗎？

- 其實未必，很多資料庫完全沒有 FK
- 沒有 FK 的資料庫，如何知道表跟表之間的關連性？如果沒有人告訴你，就只能用猜的

???

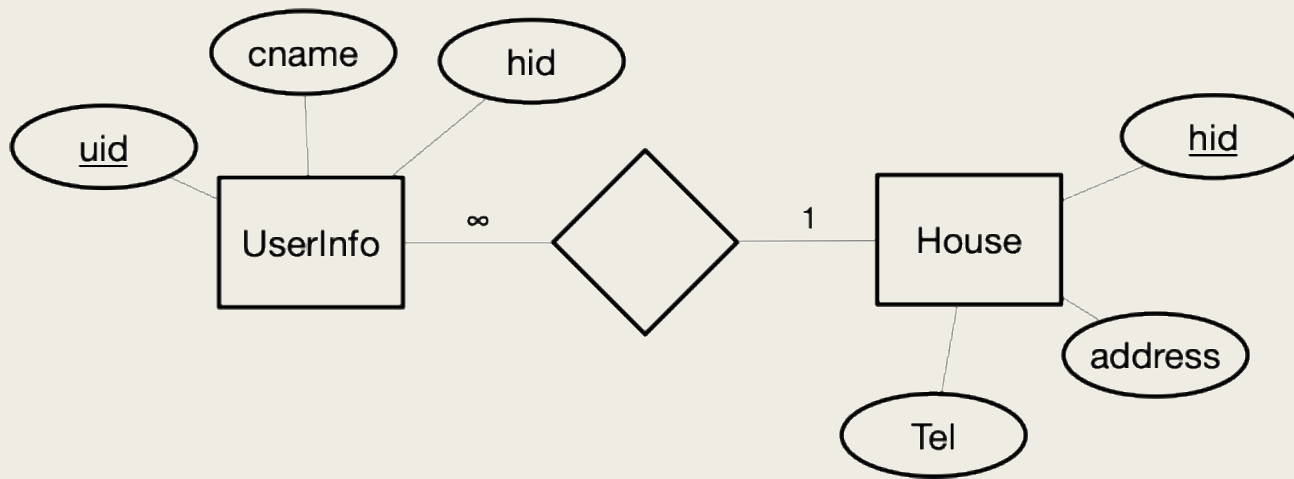
<u>a</u>	b	c
A01	王大明	1
A02	李大媽	1
A03	王小毛	1

<u>d</u>	e	f
1	台中市台灣大道一段 50 號	1111
2	台中市中山路10號	2222

g	h
1	1234
2	5678

ERD – Entity Relationship Diagram

- ER 是操作資料庫最重要的一份文件，沒有 ER，資料庫無法操作
- 有 FK 的資料庫可以自動產生 ER



UML類別圖畫法，省空間

資料字典

- ER 還需要配合資料字典才能知道每個欄位的詳細資料
- 沒有標準格式，每個單位自己決定

UserInfo 資料表

序號	說明	欄位名稱	資料型態	NULL?
*1	帳號	uid	NCHAR(10)	N
2	中文姓名	cname	NVARCHAR(50)	Y
3	住在哪個屋子裡	hid	INT	Y

正規化

■ 資料庫設計時的遵循準則

- 遵守越多資料越破碎，越少資料重複性越高
- 實務上通常遵守三個正規化：1NF、2NF與3NF

	UNF (1970)	1NF (1971)	2NF (1971)	3NF (1971)	EKNF (1982)	BCNF (1974)	4NF (1977)	ETNF (2012)	5NF (1979)	DKNF (1981)	6NF (2003)
主鍵（無重複元組）	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
沒有重複組	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
欄位原子性（元組只有一個值）	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
沒有部分函數相依（值相依於每個主鍵這一整體）	✗	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓
沒有傳遞函數相依（值僅相依於候選鍵）	✗	✗	✗	✓	✓	✓	✓	✓	✓	✓	✓
每個非平凡的函數相依涉及一個超鍵或者主鍵的子鍵	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	不適用
沒有函數相依造成的冗餘	✗	✗	✗	✗	✗	✓	✓	✓	✓	✓	不適用
每個非平凡的多值相依都有一個超鍵	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓	不適用
超鍵是每個顯式連接相依的一部分 ^[1]	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓	不適用
候選鍵隱含了每個非平凡的連接相依關係	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	不適用
每個約束都是域約束和鍵約束的結果	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	不適用
每個連接相依都是平凡的	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓

第一正規化

- 欄位內只能放一筆資料
- 例如王大明有兩棟屋子...

<u>uid</u>	cname	hid
A01	王大明	1、2

違反第一正規化

<u>uid</u>	cname	hid
A01	王大明	1
X A01	王大明	2

遵守第一正規化，但違反 PK

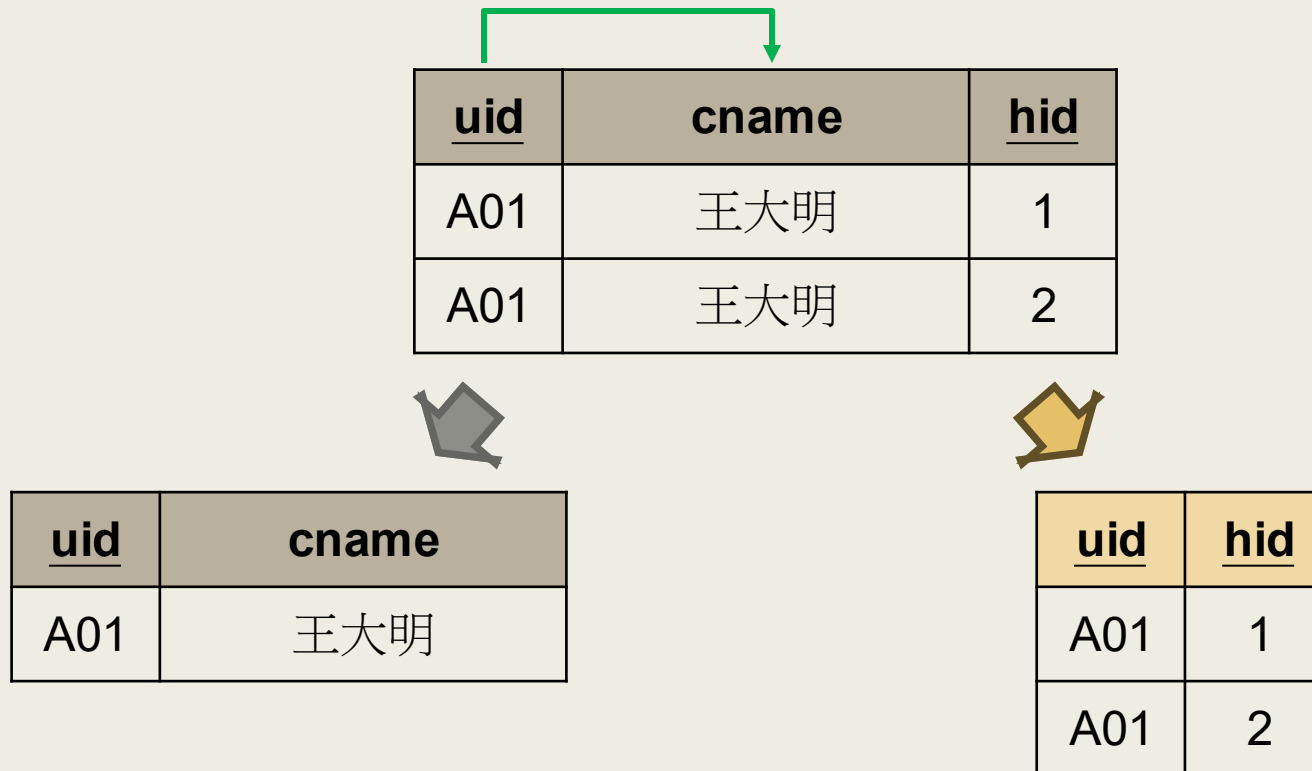
違反第一正規化結果

- hid 一定要設定 nvarchar 型態，並且欄位長度要設定最大
- 要知道 2 號屋子住了誰，非常難處理
- 王大明賣掉 5 號屋子，非常難處理
- hid 無法設定有效索引
- 違反第一正規化的資料庫雖然可以用，但處理速度極其沒有效率

<u>uid</u>	cname	hid
A01	王大明	1、2、5、23
A02	李大媽	2、23

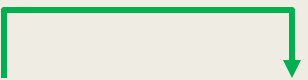
第二正規化

- 當 PK 為複合欄位時，其他欄位必須與 PK 完全相依
- 例如此資料表 uid + hid 為 PK 時，cname 只與 uid 有關，但與 hid 無關



違反第二正規化結果

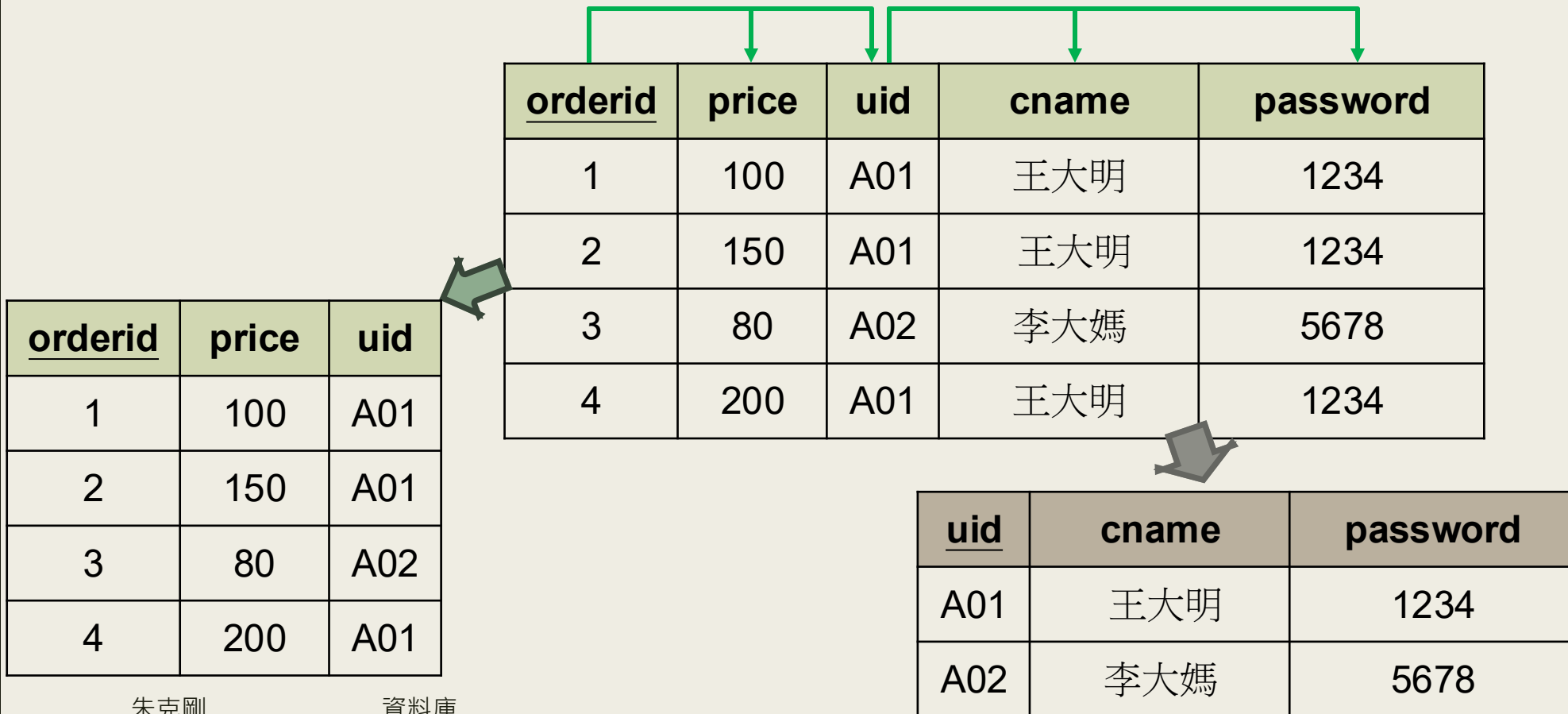
- 一個人有兩棟以上的房子時，uid 與 cname 欄位資料大量重複，浪費儲存空間
- 若 A01 改名字，所有 A01 的 cname 欄位都要全部跟著改



<u>uid</u>	cname	<u>hid</u>
A01	王大明	1
A01	王大明	2

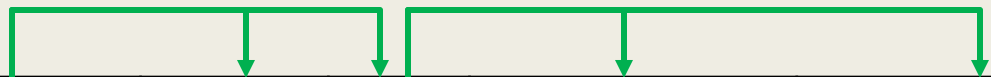
第三正規化

- 非主鍵欄位不可彼此有關，或者非主鍵欄位之間不得有傳遞相依
- 例如：帳單資料表，orderid -> uid -> cname, password



違反第三正規化結果

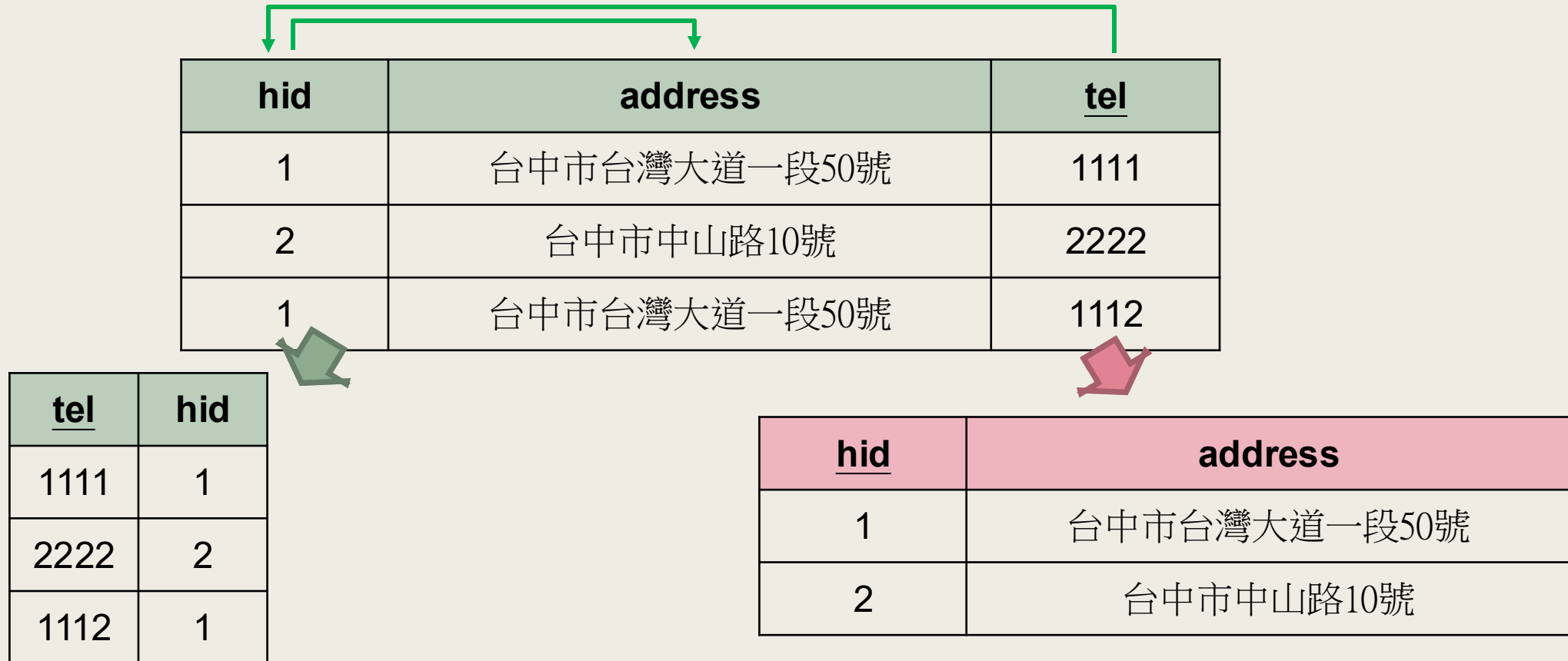
- 在 uid、cname 與 password 欄位，資料大量重複
- 如果 A01 修改了密碼，A01 的所有密碼欄位都要全部更新



<u>orderid</u>	price	uid	cname	password
1	100	A01	王大明	1234
2	150	A01	王大明	1234
3	80	A02	李大媽	5678
4	200	A01	王大明	1234

另外一個例子 1/2

- 下面這個資料表已經遵守第二正規化，且一個屋子可以有多支電話，電話為 PK
- 表中三個欄位 tel -> hid -> address 違反第三正規化



另外一個例子 2/2

- 因為 hid 與 address 互相依賴（彼此一對一對映），因此也可以是 tel -> address -> hid 這樣的依賴關係
- 同樣違反第三正規化



hid	address	<u>tel</u>
1	台中市台灣大道一段50號	1111
2	台中市中山路10號	2222
1	台中市台灣大道一段50號	1112

跟上一張投影片比，
哪一種分割方式比較好？
提示：哪一種省空間
就選哪一種



<u>tel</u>	address
1111	台中市台灣大道一段50號
2222	台中市中山路10號
1112	台中市台灣大道一段50號



<u>address</u>	hid
台中市台灣大道一段50號	1
台中市中山路10號	2

違反第三正規化結果

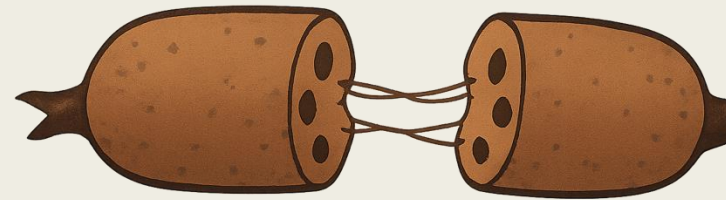
- 若一個屋子裝了多支電話時，在 address 欄位資料大量重複



hid	address	<u>tel</u>
1	台中市台灣大道一段50號	1111
2	台中市中山路10號	2222
1	台中市台灣大道一段50號	1112
1	台中市台灣大道一段50號	1113
1	台中市台灣大道一段50號	1114

關連

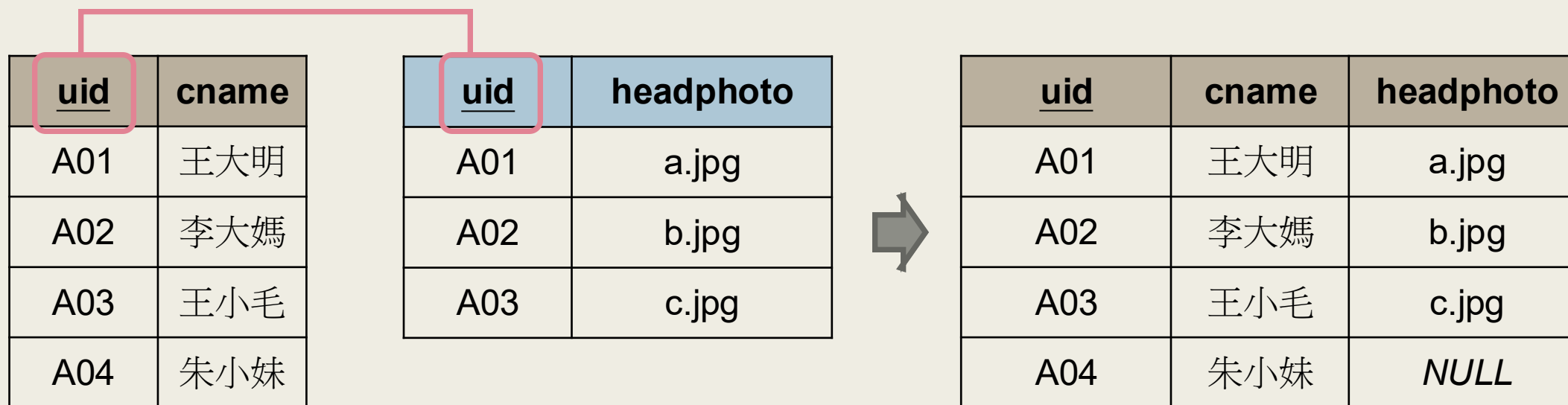
- 若兩張表的資料源自於一張表，就表示他們彼此間是有關連的，共分成三種類型：
- 一對一
 - 通常可合併成一個資料表
- 一對多、多對一
 - 兩個資料表
- 多對多
 - 三個資料表



藕斷絲連

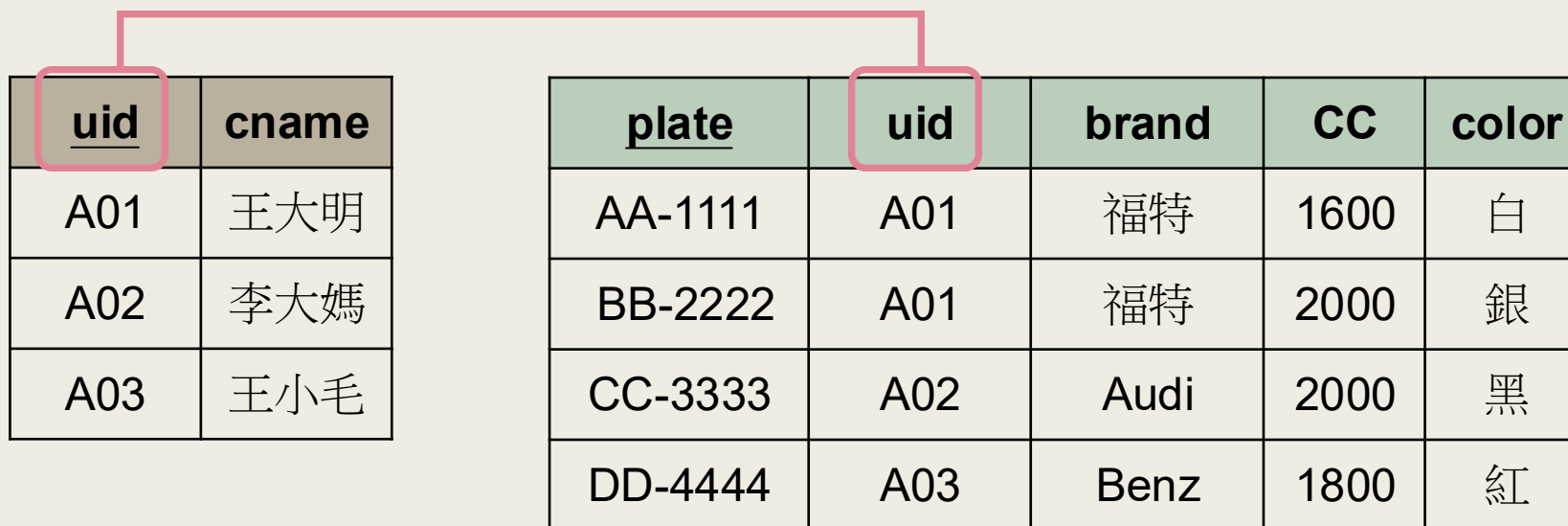
一對一關連

- 關連線兩端都是主索引，這時可以考慮合併成一張表

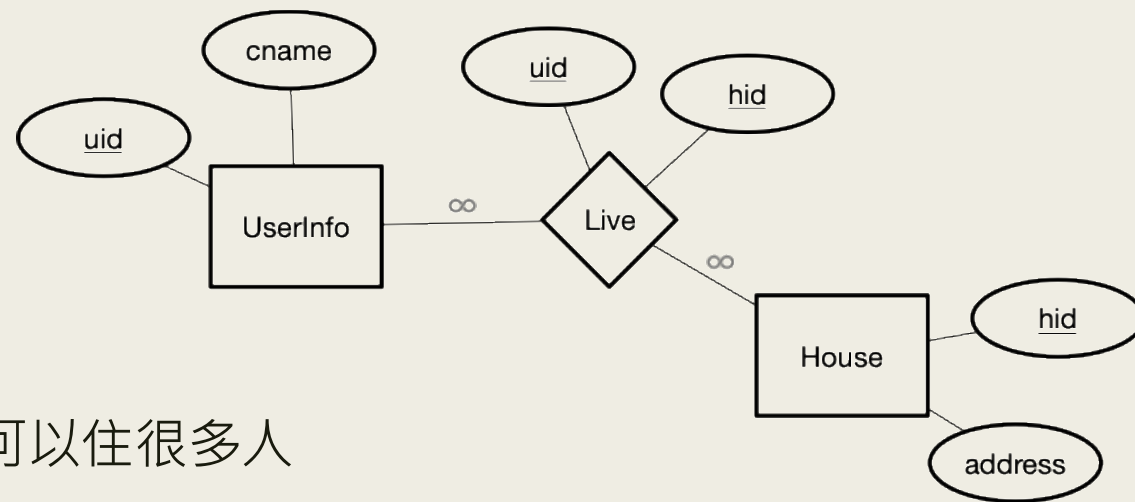


一對多、多對一關連

- 關連線一端是 PK，另一端不是，這時需要兩張表
- 一個人可以買多部車子，一部車子只會有一位車主



多對多關連



- 需要第三個資料表來記錄多對多的關連
- 一個人可以有很多棟屋子，一棟屋子中也可以住很多人

<u>uid</u>	cname
A01	王大明
A02	李大媽
A03	王小毛
A04	朱小妹

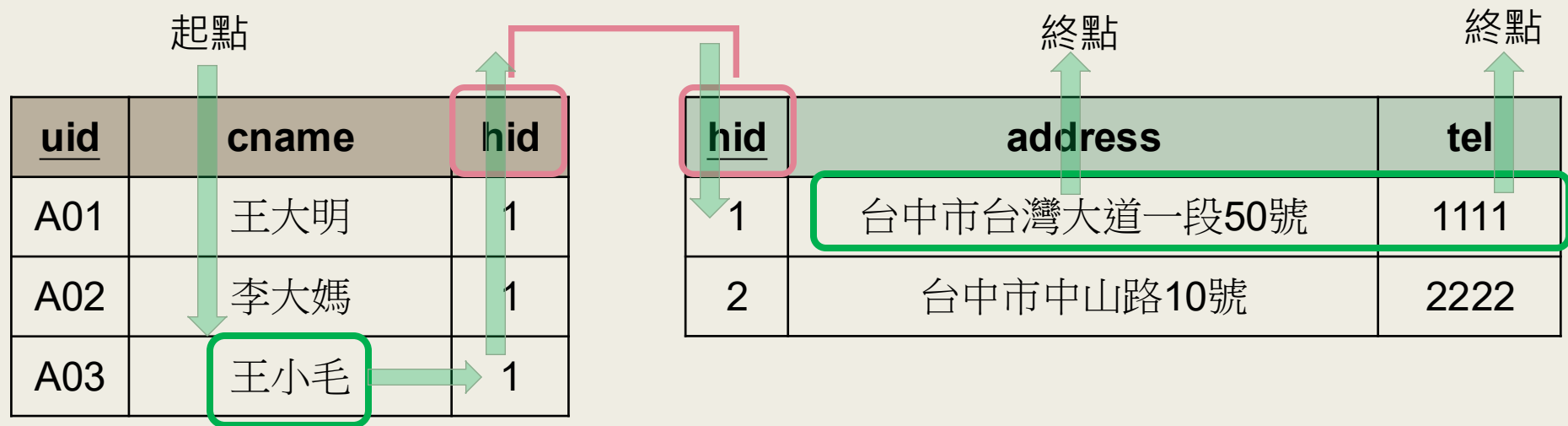
<u>uid</u>	<u>hid</u>
A01	1
A01	2
A02	1
A03	1

<u>hid</u>	address
1	台中市台灣大道一段50號
2	台中市中山路10號
3	台中市英才路30號

左右兩個資料表的 PK 為此表的 PK

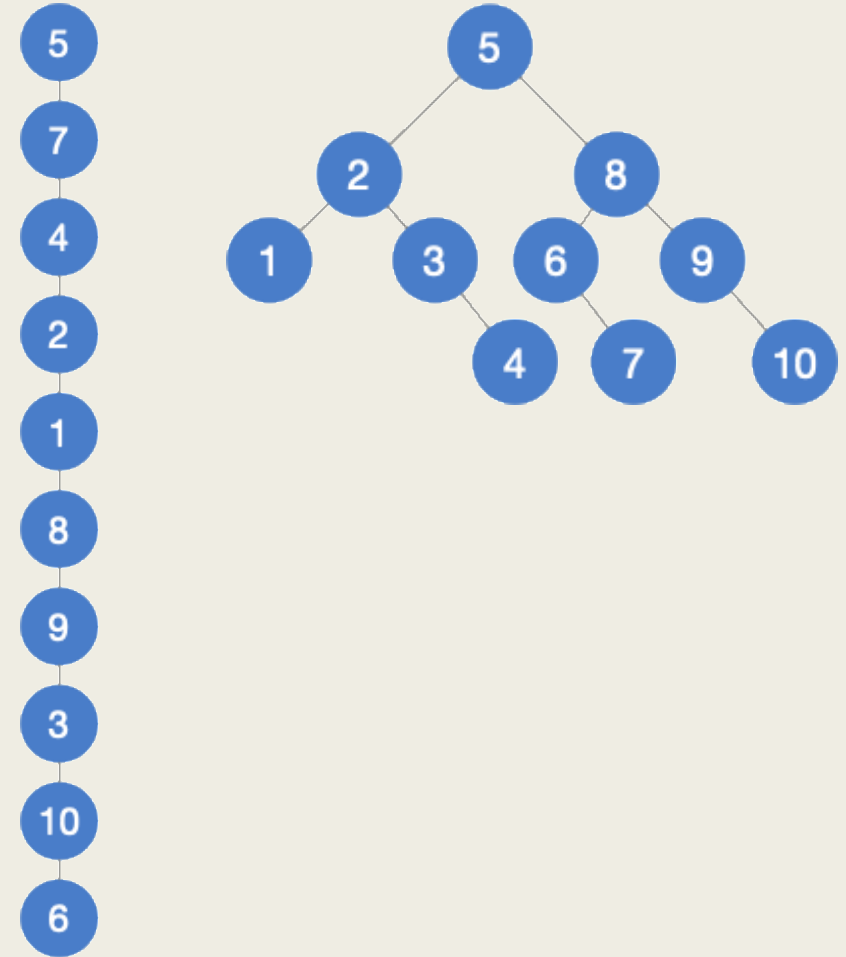
查詢軌跡

- 查詢資料的過程需要靠關連，例如：王小毛住哪以及電話為何
- 條件為搜尋起點，終點為想要看什麼欄位，透過關連找出整個軌跡需要哪些表



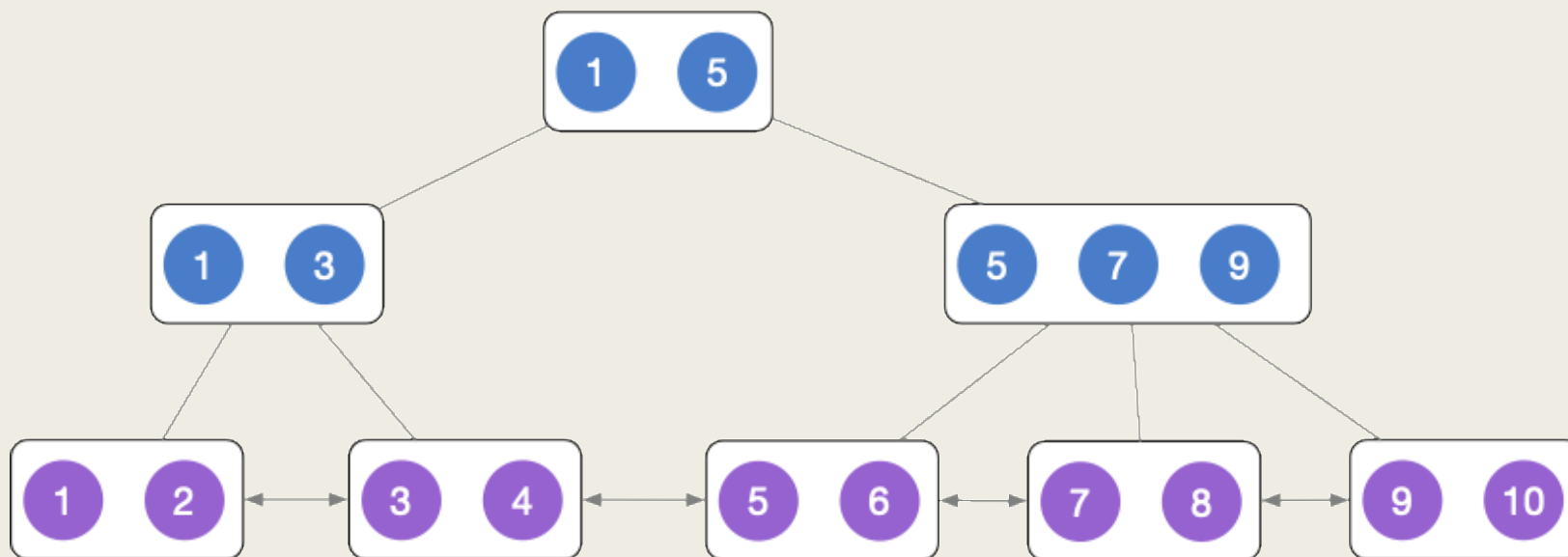
效能 - 索引 (Index)

- 目的：加快資料搜尋速度
- 原理：建立二位元搜尋樹，以硬碟空間換查詢時間
- 需要設索引的欄位：
 - 作為查詢「條件」的欄位
 - 關連線兩端的欄位
 - 作為排序的欄位
- 有設索引的稱為二位元搜尋 $O(\log n) \rightarrow \log_2 n$
 - 當資料有1,000,000,000,000 (一個兆)，搜尋次數為40 次
- 沒有設索引的稱為線性搜尋 $O(n) \rightarrow n$



B+ Tree

- Binary Tree 的進階版，專門用於資料庫索引
- Tree 的高度低，左右平衡，所有資料位於葉節點，範圍搜尋非常有效率



設定每個節點最多3筆資料 (階數為3)